AutoJudge: Predicting Programming Problem Difficulty

Submitted by:
Aditi B R (23113009)

Problem Statement

Online coding platforms host thousands of programming problems with varying difficulty levels. Accurately estimating the difficulty of a problem is essential for learners, contest organizers, and automated judging systems. However, manual labeling of problem difficulty is subjective, time-consuming, and inconsistent across platforms.
This project proposes AutoJudge, a machine learning–based system that automatically predicts:

1. The difficulty class (Easy, Medium, Hard)
2. A numerical difficulty score

using only the textual content of programming problems. The system combines classical NLP techniques with supervised machine learning models and provides an interactive web interface for real-time predictions.

Dataset Description

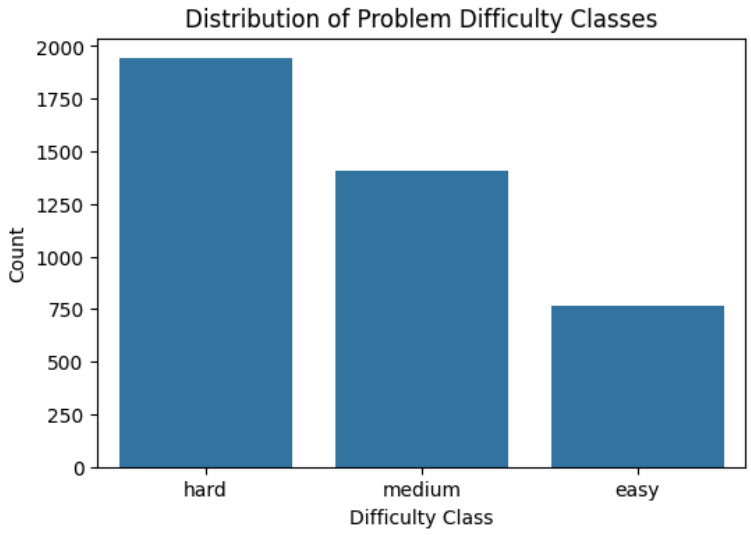
The dataset consists of programming problems with the following fields:

- Problem Description – textual explanation of the problem
- Input Description – format and constraints of inputs
- Output Description – expected output format
- problem_class – categorical difficulty label (Easy, Medium, Hard)
- problem_score – numerical difficulty score

The dataset contains no missing values, making it suitable for direct preprocessing and modeling.

Class Imbalance Visualization

- The dataset is imbalanced, with Hard problems being the most frequent.
- Easy problems are comparatively fewer.



Data Preprocessing

1. Combining Text Fields

To capture full semantic information, all textual fields were combined into a single feature:
full_text = problem_description + input_description + output_description
This ensures the model learns from the entire problem context.

2. Text Cleaning

The combined text underwent the following cleaning steps:

- Removal of line breaks (\n, \t)
- Removal of extra whitespaces
- Conversion to lowercase
- Removal of punctuation and special characters

3. Text Normalization

To improve feature quality:

- Tokenization – split text into words
- Stopword removal – removed common words
- Lemmatization – reduced words to base form

4. Label Encoding

Class	Encoded Value
Easy	0
Medium	1
Hard	2

The Target Variable problem_class was label encoded. The trained Label encoder was saved and reused during inference.

5. Numerical Feature Standardization

For regression, numerical features were standardized using StandardScaler to ensure consistent scale and better convergence.

6. Feature Extraction using TF-IDF

To convert text into numerical vectors, TF-IDF (Term Frequency–Inverse Document Frequency) was used.

TF-IDF Configuration

- Maximum features: 5000
- N-gram range: (1, 2)

This configuration captures:

- Important keywords (unigrams)
- Meaningful phrases (bigrams)

TF-IDF assigns higher importance to terms that are frequent in a document but rare across the corpus, making it suitable for difficulty estimation.

Classification Model

Model	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Logistic Regression	0.538	0.52	0.46	0.48
XGBoost Classifier	0.539	0.50	0.46	0.47
Random Forest	0.549	0.52	0.42	0.40

Model Selection

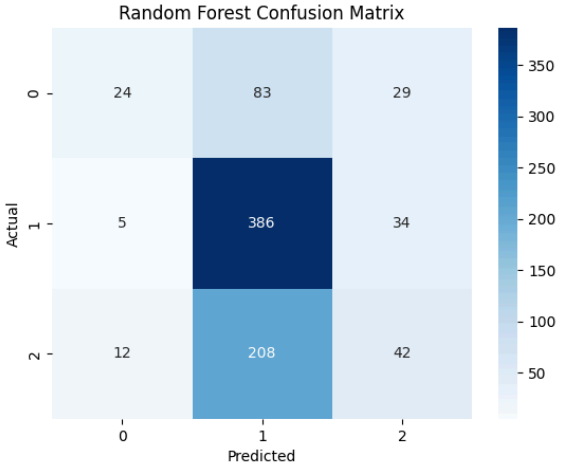
- Random Forest effectively captures non-linear relationships between TF-IDF features, text length, and keyword-based features.
- It is robust to feature noise and overfitting due to ensemble averaging.
- The model provided the best trade-off between accuracy and generalization across difficulty classes.
- Given the goal of reliable difficulty classification rather than marginal metric gains, Random Forest was selected as the final classification model.

Results

```
Random Forest Accuracy: 0.5492102065613609
              precision    recall  f1-score   support

           0       0.59      0.18      0.27       136
           1       0.57      0.91      0.70       425
           2       0.40      0.16      0.23       262

    accuracy                           0.55       823
   macro avg       0.52      0.42      0.40       823
weighted avg       0.52      0.55      0.48       823
```



Overall Accuracy
The Random Forest classifier achieves an overall accuracy of 54.92%, meaning that approximately 55 out of 100 problems are classified correctly.
Although this accuracy may appear moderate, it is important to consider that:

- Problem difficulty classification is inherently subjective
- Difficulty labels are based on human judgment
- The model relies only on textual descriptions, without access to solution logic, code complexity, or execution constraints

Given these limitations, the obtained accuracy is reasonable and acceptable for a text-based multi-class difficulty prediction task.

The model performs best on Medium difficulty problems. This is primarily because:

- Medium problems form the largest class in the dataset
- The model has more samples to learn recurring textual patterns such as:
  - Common algorithms
  - Standard problem structures
  - Familiar competitive programming terminology

The high recall of 0.91 for the Medium class indicates that the majority of actual Medium problems are correctly identified.

Confusion Matrix Analysis

The confusion matrix reveals that:

- Most misclassifications occur between adjacent difficulty levels:
  - Easy ↔ Medium
  - Medium ↔ Hard
- Very few Easy problems are directly classified as Hard, and vice versa

This behavior is desirable and indicates that the model has learned the relative ordering of difficulty levels, even when exact classification is incorrect.

Results Interpretation

- The low MAE indicates that, on average, the predicted difficulty score deviates only slightly from the true value.
- The RMSE being close to MAE suggests the model does not suffer from extreme outliers or unstable predictions.
- The model demonstrates consistent performance across different difficulty levels, making it suitable for real-world use cases such as automated problem evaluation.

Regression Model

Model	R^2 Score	MAE	RMSE
Linear Regression	-2.356	3.213	4.014
Random Forest Regressor	0.136	1.693	2.037
Gradient Boosting Regressor	0.154	1.676	2.016

Model Selection

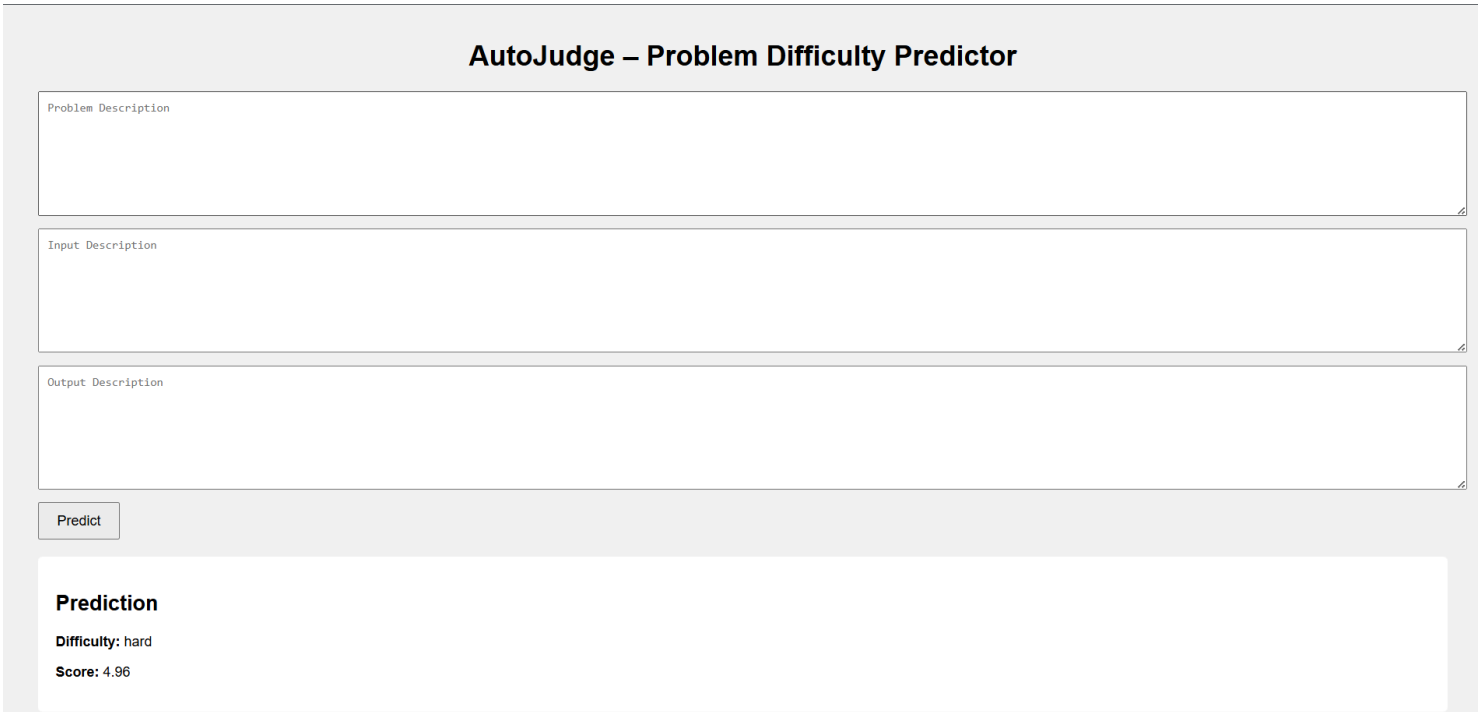
- Multiple regression models were evaluated, including:
  - Linear Regression
  - Random Forest Regressor
  - Gradient Boosting–based models
- A tree-based ensemble regressor was selected due to its ability to:
  - Capture non-linear relationships
  - Handle high-dimensional sparse TF-IDF features
  - Maintain robustness against noise in text data

```
R² Score: 0.15354335412811537
MAE: 1.6759275548072607
RMSE: 2.015737830206395
```

Results

- R^2 Score (0.15): The model explains about 15% of the variance in difficulty scores, which is expected due to the subjective and noisy nature of manual scoring based only on text.
- MAE (1.68): On average, predictions deviate by ~1.7 points, indicating reasonably accurate difficulty estimation.
- RMSE (2.02): Close to MAE, showing stable predictions with no major outliers.
- Overall: The Random Forest regressor captures non-linear patterns from textual features and provides consistent, practical score predictions despite limited information.

Web Interface



A web-based interface was developed to provide an intuitive and user-friendly way to interact with the trained machine learning models. The interface allows users to input problem details and receive instant predictions for both problem difficulty class and difficulty score.

Technology Stack

- Backend: Flask (Python)
- Frontend: HTML, CSS
- Model Integration: Pre-trained classification and regression pipelines loaded using joblib

Input Components

The interface provides three text input fields:

1. Problem Description – General description of the programming problem
2. Input Description – Explanation of input format and constraints
3. Output Description – Explanation of expected output

These fields directly correspond to the text features used during model training, ensuring consistency between training and inference.

Backend Processing

- Upon form submission:
  - The input text fields are combined into a single text input.
  - The same preprocessing steps used during training (TF-IDF transformation and feature extraction) are applied via the saved pipeline.
- The processed input is passed to:
  - The classification model to predict the difficulty class (Easy / Medium / Hard)
  - The regression model to predict a continuous difficulty score

Output Display

- The predicted difficulty class and score are displayed dynamically on the same page.
- The result section appears only after submission, avoiding unnecessary clutter.

Conclusion

- This project presents an automated approach to predict the difficulty of programming problems using machine learning and natural language processing techniques. Textual information from problem statements was preprocessed and transformed using TF-IDF features, enabling effective representation of semantic content.
- Among the evaluated classification models, Random Forest demonstrated the most balanced performance compared to Logistic Regression and XGBoost, particularly in handling non-linear relationships and class variability. The regression model achieved reasonable error values when predicting continuous difficulty scores, indicating stable and consistent performance.
- A user-friendly web interface was successfully developed using Flask, HTML, and CSS, allowing real-time difficulty prediction based on user input. Overall, the system validates the feasibility of automated difficulty assessment and provides a strong baseline for future enhancements.